Undersigned: TF meeting w/ Alyssa

Fri 30 Oct · MS3 due Thu 12 Nov, 10:00 PM ET, `milestone3` branch · numbers from `docs/linker-results.md`

Repo: milestone3 branch · README MS3 section
Results: linker-results.md · full report
W&B: project · current model run · pipeline run
Vertex: training jobs · pipeline runs
Cloud Run: undersigned-api · request logs
Evidence: training · Cloud Run checks

1 What MS3 was supposed to do

> MS2 could only place comments that cite a section number, which is **3.7%** of them. For MS3 we trained a model to link the rest: given a passage, rank the rule's sections by which one it's arguing about. Then train it on Vertex, track in W&B, and put it behind the API on Cloud Run.

Training labels come from the comments that do cite: we hide the citation and make the model recover it. No hand labelling.

2 Where we are vs the targets we set in MS2

Target	Goal	Now	
Linking accuracy (right section first)	≥ 69%, aim 75%	**69.8%** on the 232 passages the LLMs also answered; 66.7% on all 490	**min. met**
Cost of reading at scale	≤ $5 per 1,000 passages, keeping ≥ 94% of the large model's accuracy	Cascade: **72.0% at $2.43**, 95% of Sonnet's accuracy	**met**
Works on uncited passages	the 96% that don't cite	90.0% on 40 hand-labelled uncited passages; Sonnet 87.5%	**good so far**
Serving cost per request	no LLM call at request time	ONNX model on Cloud Run, ~172 ms median	**met**

For scale: when two of us labelled the same 30 passages we only agreed on 21 (70%). ~17% of citation labels can't be recovered from the text at all, so ~83% is roughly the ceiling. The right section is in our top 3 82.7% of the time.

3 Comparison with Sonnet

232 test passages	Top-1	$ / 1,000
Untuned encoder (MS2)	44.8%	0
Hybrid search	53.4%	0
Ours (fine-tuned bge-small)	**69.8%**	**0**
Haiku	69.0%	1.29
Sonnet	75.4%	19.58
Ours, least confident ones sent to Sonnet	**72.0%**	**2.43**

We don't beat Sonnet: −5.6 pts (CI −11.9 to +0.5). vs Haiku: +0.9.

Points worth making

- **Cost.** The 7 test dockets have 721k passages. Sonnet on all of them is ~**$14k**.
- **Our confidence is useful.** On our most confident 20% we're better than Sonnet (96% vs 87%); on our least confident 20% much worse (33% vs 67%). That's why sending only the unsure ones to Sonnet works.
- **New rules.** Trained with no Title IX comments at all, just the rule's preamble text, we get **67% on Title IX** (was 50%, hybrid search 57%). Only tested on 1 rule so far.
- **Same model size as MS2**, so no new service; it just replaced the old one.

Weak spots

Small dockets	MSHA 60%, lead & copper 52.5%, Clean Power Plan 47%. Fewest citing comments, shortest preambles
Neighbouring sections	Most errors pick a section next to or cross-referenced by the right one. These get flagged `needs_reader`
Transfer, n=1	Title IX result is one held-out rule. 5 more to run

4 MS3 checklist

Requirement	Status	What we have	Where
1 · Advanced training workflows			
Experiment tracking (W&B)	done	Loss, LR, val accuracy per epoch, test predictions table. Project `undersigned-linker`	runs `ls2ekysw` (current), `zyqws7hu`
Serverless training on Vertex AI, logged and reproducible	done	Vertex and laptop runs gave identical ranks on all 490 test passages	`linker-train/vertex/`: `submit.sh`, `job.yaml`
Multi-GPU (if applicable)	pending	T4 quota was 0, requested 2 on 29 Oct. Everything so far on CPU	
2 · Serverless deployment			
Model/API on Cloud Run	done	`undersigned-api`, 2 CPU / 4 GiB, scales to zero. New model deployed 29 Oct	`api-service/cloudrun/`
Public or staff-accessible	ask	private right now, see Q1	
Deploy config + how to redeploy	done	one script, pulls the promoted model from the bucket and deploys	`deploy.sh`, README "Deployment"
3 · ML workflow with Vertex AI			
Automated pipeline: preprocessing, training, evaluation	done	**train** → **evaluate** → **promote**, only if it beats hybrid search and the current model. The check caught a rounding bug on the first run	`vertex/pipeline.py`, `promote.py`
4 · Production monitoring			
Logging, error tracking, metrics, documented	done	JSON request logs in Cloud Logging, `/stats` with error counts and latency percentiles	README "Deployment and monitoring"
5 · Working API			
Exposes predictions	done	`POST /link` returns ranked sections + a `needs_reader` flag	README "The API"
Documented and tested	done	README has fields + example curl. 44 API tests, run in CI	`api-service/tests/`, `scripts/link_check.py`
Deliverables			
README: reproduce, deploy, pipeline, API; changes since MS2 highlighted	done	"Milestone 3: the linker" at the top	`README.md`
Screenshots: W&B run, `curl` to the deployed service	to do	only have text logs so far	`docs/evidence/`
15-min slides; everyone can answer on every part	to do	she'll ask each of us about any part of the repo	

5 Questions for Alyssa

1. Is it OK to keep Cloud Run private and give staff access (`run.invoker`)? Whose accounts?
2. If the GPU quota doesn't come through, is the CPU Vertex run fine? Do we need multi-GPU at all?
3. The Sonnet cascade is evaluated on saved Sonnet outputs. Does it need to be live in the API for MS3?
4. Is it a problem that we're below Sonnet if the argument is cost + it works on unlabelled rules?
5. How many held-out rules would she want to see for the Title IX-style result?
6. How much of the 15 min should be demo vs results?

6 To do before 12 Nov

Task	Who
Screenshots: W&B run, curl to Cloud Run	
Add the preamble + letter scores to `/link` (API serves the model alone, 65.5%)	
Held-out runs for the other 5 rules	
Add `linker-train` to the CI matrix	
Slides	